**DIGITAL DESIGN REPORT**

# 4-Bit RISC CPU Datapath with 3-Stage Pipeline (IF – ID – EX)

Vivado 2026.1 Course Project



| | |
|---|---|
| **Student Name:** | Tran Quan Bao |
| **Roll Number:** | SE190570 |
| **Target Environment:** | AMD Xilinx Vivado 2026.1 / Artix-7 (xc7a35tcpg236-1) |
| **Verification Result:** | **88 / 88 Automated Simulation Checks PASSED (100%)** |



## Project Overview

This technical report presents in detail the design, implementation, and comprehensive verification of a 4-Bit RISC CPU Datapath featuring a 3-stage pipeline architecture (IF – ID – EX) using Verilog HDL, targeted for synthesis and implementation on the AMD Xilinx Vivado 2026.1 platform.

The processor is built strictly adhering to standard architectural specifications:

- **Datapath:** 4-bit data width, operating on 4-bit unsigned and signed integer representations.
- **Instruction Set Architecture (ISA):** Fixed-width 12-bit RISC architecture with compact opcode and operand field encoding.
- **Register File:** Four 4-bit general-purpose registers (R0–R3), supporting dual parallel asynchronous read ports and a single synchronous write port.
- **Memory Architecture:** Harvard architecture with physically separated memories: Instruction Memory (ROM) of 16 words × 12-bit and Data Memory (RAM) of 16 words × 4-bit.
- **Execution Performance:** Throughput of 1 instruction per clock cycle (Throughput = 1 CPI once the pipeline is primed).
- **Branch Hazard Handling:** Control hazards caused by branch instructions (JMP, JZ) are fully resolved via an automatic 2-bubble flush mechanism at the EX stage.
- **Verification Evaluation:** Complete functional verification was achieved with 88/88 automated simulation checks PASSED (100%), including 44 module-level test cases and 44 system-level integration test cases.

## Part A. Design Architecture & Module Hierarchy

The hardware architecture of the microprocessor is structured hierarchically into the top-level module cpu_top alongside 10 specialized sub-modules distributed across three pipeline stages:

- **cpu_top (Top-Level Datapath):** The top-level integration entity that interconnects internal data buses and control signals across the three pipeline stages (IF, ID, EX).
- **program_counter (PC - Stage 1 IF):** 4-bit register holding the memory address for instruction fetch. Operates synchronously on the rising clock edge (**pc_en = 1**), supporting sequential increment (PC + 1) and target branch loading from the EX stage.
- **instruction_memory (ROM - Stage 1 IF):** Program storage unit sized at 16 words × 12-bit, initialized with machine code via **$readmemh** from a text file (**.mem**). Unused memory locations are padded with NOP instructions (**12'hC00**) to eliminate undefined states (X).
- **instruction_register (IF/ID Pipeline Register):** Boundary pipeline register between the IF and ID stages. Latches the 12-bit instruction word (IR), corresponding PC address, and **valid_out** control flag. Upon branch flush (**flush = 1**) or system reset, it automatically inserts a NOP bubble (**valid_out = 0**).
- **control_unit (Stage 2 ID & Stage 3 EX):** Hardwired control unit composed of two sub-blocks: • **instruction_decoder** (ID): Decodes opcode [11:8] into 9 control signals with safe default values. • **branch_unit** (EX): Evaluates branch conditions (**is_jmp, is_jz**) alongside the Zero flag to resolve branching (**pc_sel = 1**) and generate the pipeline flush signal (**flush = 1**).
- **pipeline_register (ID/EX Pipeline Register):** Boundary register between ID and EX stages. Stores all 9 control signals, source/destination register addresses (Rd, Rs), 4-bit immediate field (imm[3:0]), PC address, and execution control bits.
- **register_file (RF - Stage 3 EX):** 4-bit register file containing 4 registers (R0, R1, R2, R3). Features two asynchronous combinational read ports (Port A for Rd, Port B for Rs) and a synchronous write port triggered by clock edge on **reg_we**.
- **alu (ALU Core - Stage 3 EX):** 4-bit arithmetic and logic core. Integrates a shared **adder_subtractor** (following ARM convention: Carry = no-borrow on subtraction), a **logic_unit** (AND, OR, XOR, NOT, PASS), and a **zero_detector**.
- **flag_register (Stage 3 EX):** Stores two condition flags: Zero (Z) and Carry (C). Features independent write-enable lines (**zero_we, carry_we**) ensuring logical operations update Zero without modifying Carry.
- **data_memory (DMEM - Stage 3 EX):** 16 words × 4-bit data RAM. Supports asynchronous combinational read for **LOAD** operations and synchronous write on rising clock edge gated by **mem_we** for **STORE** operations.
- **mux2 (x3 Multiplexers):** Three 4-bit 2-to-1 multiplexers managing primary routing: • PC MUX (selects between PC + 1 and target branch imm[3:0]), • Operand MUX (selects ALU operand B between Rs and imm[3:0]), • Write-back MUX (selects Rd write-back between ALU result and RAM output).

## Part B. Testbench & Verification Methodology

The verification framework was systematically implemented using a two-tier industry-standard methodology to ensure complete code coverage and functional coverage:

### 1. Module-Level Verification Suite (44 Checks)

- **tb_alu.v (11 checks):** Exhaustively validates 11 scenarios including addition, subtraction (with full verification of Carry and Borrow under the ARM no-borrow convention), logical operators (AND, OR, XOR, NOT), bypass PASS function, and Zero flag detection.
- **tb_register_file.v (11 checks):** Verifies synchronous reset, independent read/write operations across registers R0 through R3, dual-port simultaneous reads in the same clock cycle, and data preservation when **reg_we = 0**.
- **tb_control_unit.v (22 checks):** Validates the 11 control signals against truth tables across all 16 opcode encodings (including 4 reserved NOP spaces) and verifies 6 distinct branching scenarios within **branch_unit**.

### 2. CPU System-Level Verification Suite (tb_cpu.v – 44 Checks)

- **Test 1 (program.mem):** Tests immediate loading (LDI), arithmetic addition, logical XOR, and infinite JMP loop operation.
- **Test 2 (prog_arith.mem):** Validates end-to-end arithmetic with data memory interactions: LDI, ADD, SUB, Carry flag, memory STORE, memory LOAD, and Zero flag assertions.
- **Test 3 (prog_logic.mem):** Tests inter-register data movement (MOV), logical operations (AND, OR, XOR, NOT), and register clearing via **XOR Rd, Rd**.
- **Test 4 (prog_jump.mem):** Tests PC sequential increment, unconditional JMP, conditional JZ (for both Z = 1 and Z = 0), and strictly verifies pipeline **2-bubble flush** execution to prevent stale instruction execution.
- **Test 5 (Reset Verification):** Validates synchronous reset assertion and executes an XOR-check across all system buses to ensure zero occurrence of undefined (X) or high-impedance (Z) states.

## Part C. Instruction Set, Truth Tables & Execution Trace

### 1. 12-Bit Instruction Format

All processor instructions have a fixed width of 12 bits, partitioned into 4 functional bitfields:

| Bit [11:8] (4 bits) | Bit [7:6] (2 bits) | Bit [5:4] (2 bits) | Bit [3:0] (4 bits) |
|---|---|---|---|
| Opcode (Operation Code) | Rd (Destination / Src 1) | Rs (Source 2) | imm / addr (Immediate / Address) |

Register Identifiers: R0 = 2'b00 (0x0) | R1 = 2'b01 (0x1) | R2 = 2'b10 (0x2) | R3 = 2'b11 (0x3)

### 2. Instruction Set Architecture (ISA)

| Opcode | Mnemonic | Syntax | Functional Operation | Flags Updated |
|---|---|---|---|---|
| 0000 | ADD | ADD Rd, Rs | Rd ← Rd + Rs | Zero, Carry |

| | | | | |
|---|---|---|---|---|
| 0001 | SUB | SUB Rd, Rs | Rd ← Rd - Rs (Carry = no-borrow) | Zero, Carry |
| 0010 | AND | AND Rd, Rs | Rd ← Rd & Rs | Zero |
| 0011 | OR | OR Rd, Rs | Rd ← Rd \| Rs | Zero |
| 0100 | XOR | XOR Rd, Rs | Rd ← Rd ^ Rs | Zero |
| 0101 | NOT | NOT Rd | Rd ← ~Rd | Zero |
| 0110 | LDI | LDI Rd, imm | Rd ← imm[3:0] | Unchanged |
| 0111 | MOV | MOV Rd, Rs | Rd ← Rs | Unchanged |
| 1000 | LOAD | LOAD Rd, addr | Rd ← DMEM[addr] | Unchanged |
| 1001 | STORE | STORE Rd, addr | DMEM[addr] ← Rd | Unchanged |
| 1010 | JMP | JMP addr | PC ← addr | Unchanged |
| 1011 | JZ | JZ addr | If Zero = 1: PC ← addr; else continue | Unchanged |
| 1100..1111 | NOP | (Reserved) | No Operation performed | Unchanged |

### 3. Control Unit Decoder Truth Table

| Opcode | Instruction | alu_ctrl | alu_src | wb_sel | reg_we | mem_we | z_we | c_we | is_jmp | is_jz |
|---|---|---|---|---|---|---|---|---|---|---|
| 0000 | ADD | 000 (ADD) | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 0001 | SUB | 001 (SUB) | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 0010 | AND | 010 (AND) | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0011 | OR | 011 (OR) | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0100 | XOR | 100 (XOR) | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 0101 | NOT | 101 (NOT) | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0110 | LDI | 110 (PASS) | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0111 | MOV | 110 (PASS) | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1000 | LOAD | 110 (PASS) | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1001 | STORE | 110 (PASS) | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1010 | JMP | 000 (NOP) | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1011 | JZ | 000 (NOP) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 1100..1111 | NOP | 000 (NOP) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

### 4. Step-by-Step Execution Trace

| Instruction | Hex Code | Operand Values | Output Result | Flags (Z, C) | System Impact |
|---|---|---|---|---|---|
| LDI R1, 5 | 0x645 | imm = 5 (0101) | R1 = 5 (0101) | Unchanged | Loads immediate 5 into R1 |
| LDI R2, 3 | 0x683 | imm = 3 (0011) | R2 = 3 (0011) | Unchanged | Loads immediate 3 into R2 |
| ADD R1, R2 | 0x060 | A = 5, B = 3 | R1 = 8 (1000) | Z = 0, C = 0 | Addition without overflow |
| STORE R1, 0 | 0x940 | R1 = 8, addr = 0 | DMEM[0] = 8 | Unchanged | Writes value 8 to RAM address 0 |
| SUB R1, R2 | 0x160 | A = 5, B = 3 | R1 = 2 (0010) | Z = 0, C = 1 | Subtraction without borrow (5 >= 3) |

| | | | | | |
|---|---|---|---|---|---|
| LOAD R0, 0 | 0x800 | addr = 0 | R0 = 8 (1000) | Unchanged | Reads RAM[0] into R0 |
| SUB R2, R2 | 0x1A0 | A = 3, B = 3 | R2 = 0 (0000) | Z = 1, C = 1 | Result equals 0, sets Z = 1 |
| JZ 9 | 0xB09 | addr = 9, Z = 1 | PC ← 9 | Unchanged | Branch taken, flushes 2 instructions |

## Part D. Tool Flow Evidence & Screenshots (F1 – F6)

Technical verification artifacts extracted directly from the AMD Xilinx Vivado 2026.1 design environment:

### F1. Sources Window – Design Hierarchy & Simulation Sources

The project hierarchy displays the top-level module cpu_top (cpu_top.v) comprising all instantiated sub-modules: PC_REG (**program_counter**), IMEM (**instruction_memory**), IR (**instruction_register**), CU (**control_unit**), ID_EX (**pipeline_register**), RF (**register_file**), ALU_CORE (**alu**), FLAGS (**flag_register**), DMEM (**data_memory**), and 3 instances of mux2 (**PC_MUX, OPERAND_MUX, WB_MUX**). The Simulation Sources directory holds 4 testbenches and 4 memory initialization (**.mem**) files.

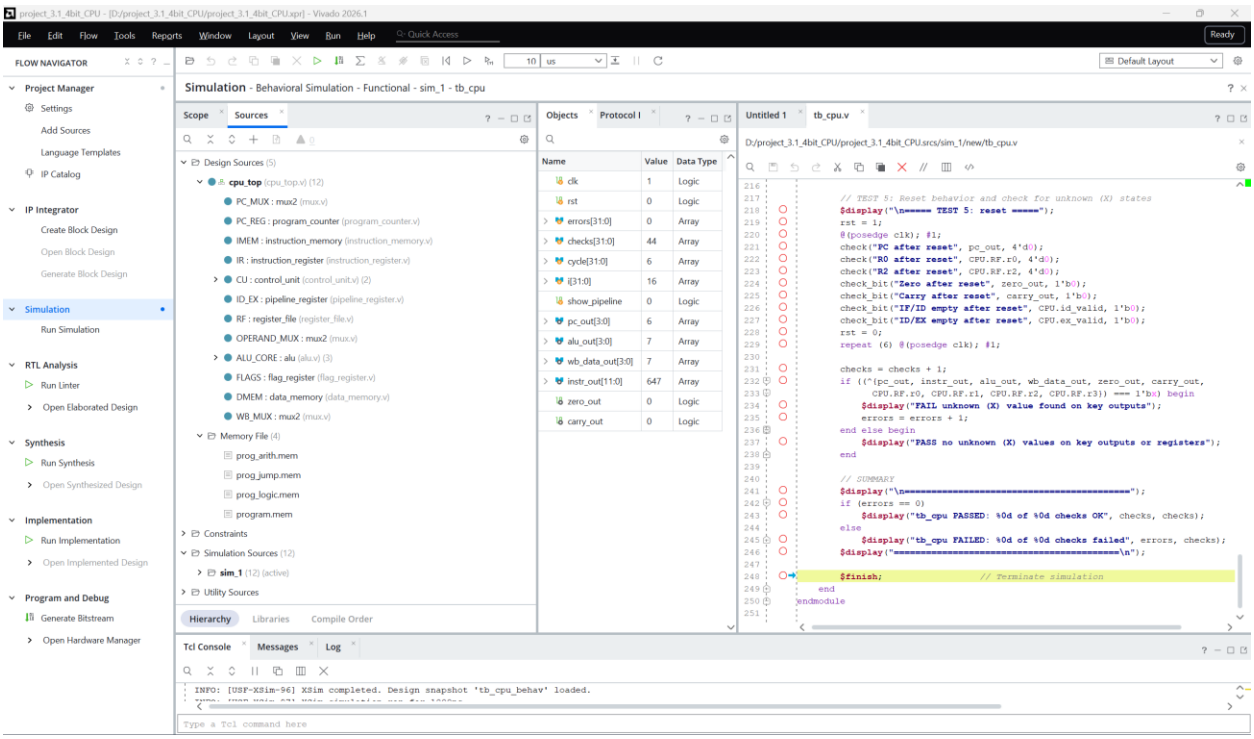


**Figure 1: Sources Window – Design Hierarchy vs Simulation Sources**

### F2. Behavioral Simulation Waveform

Real-time behavioral simulation waveform demonstrating execution across the 3 pipeline stages (IF – ID – EX). The waveform clearly indicates PC sequential incrementing, flush pulse assertion upon branch detection, and accurate register write-backs (R1–R3) synchronized to clock edges.

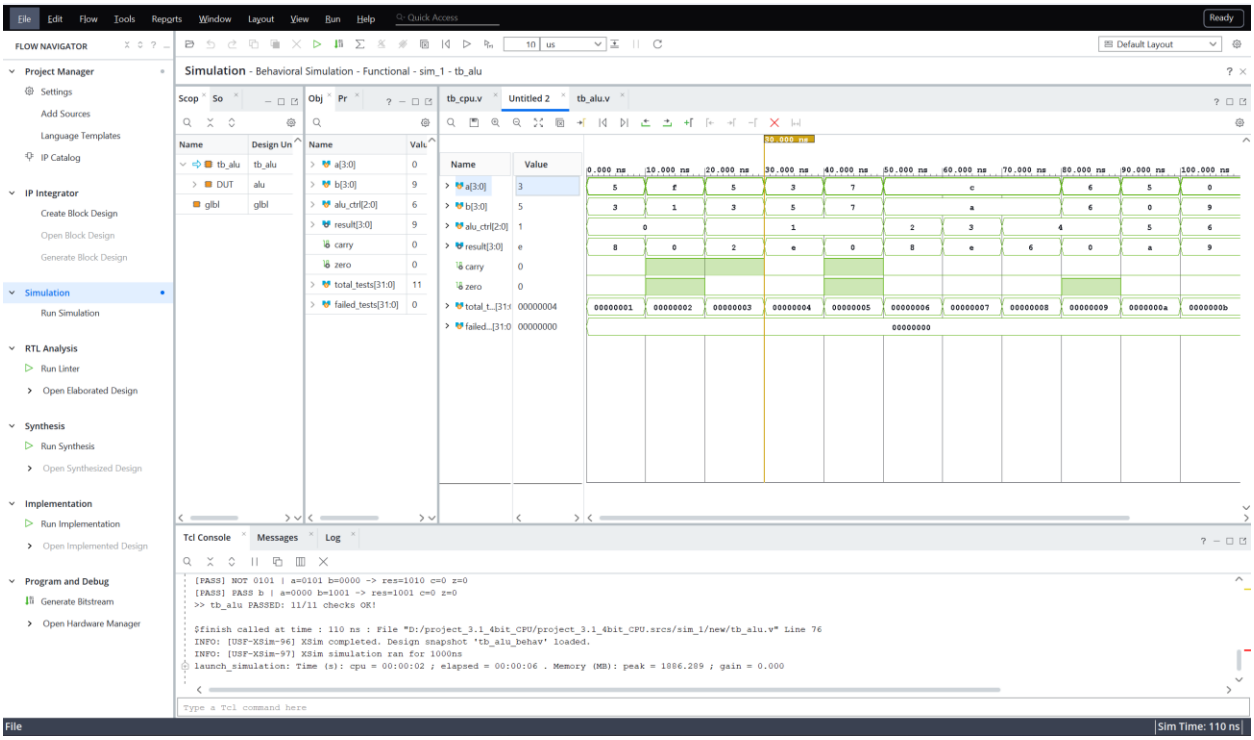



**Figure 2: Waveform - tb_alu**

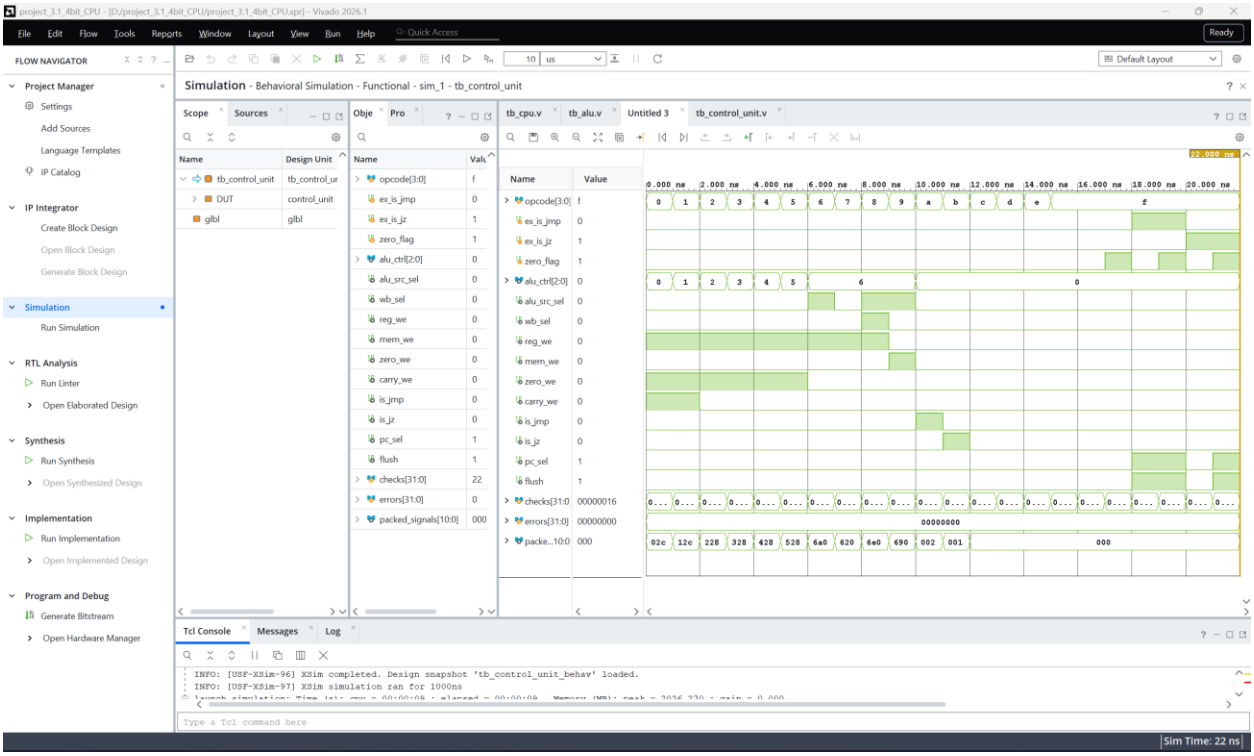



**Figure 3: Waveform - tb_control_unit**

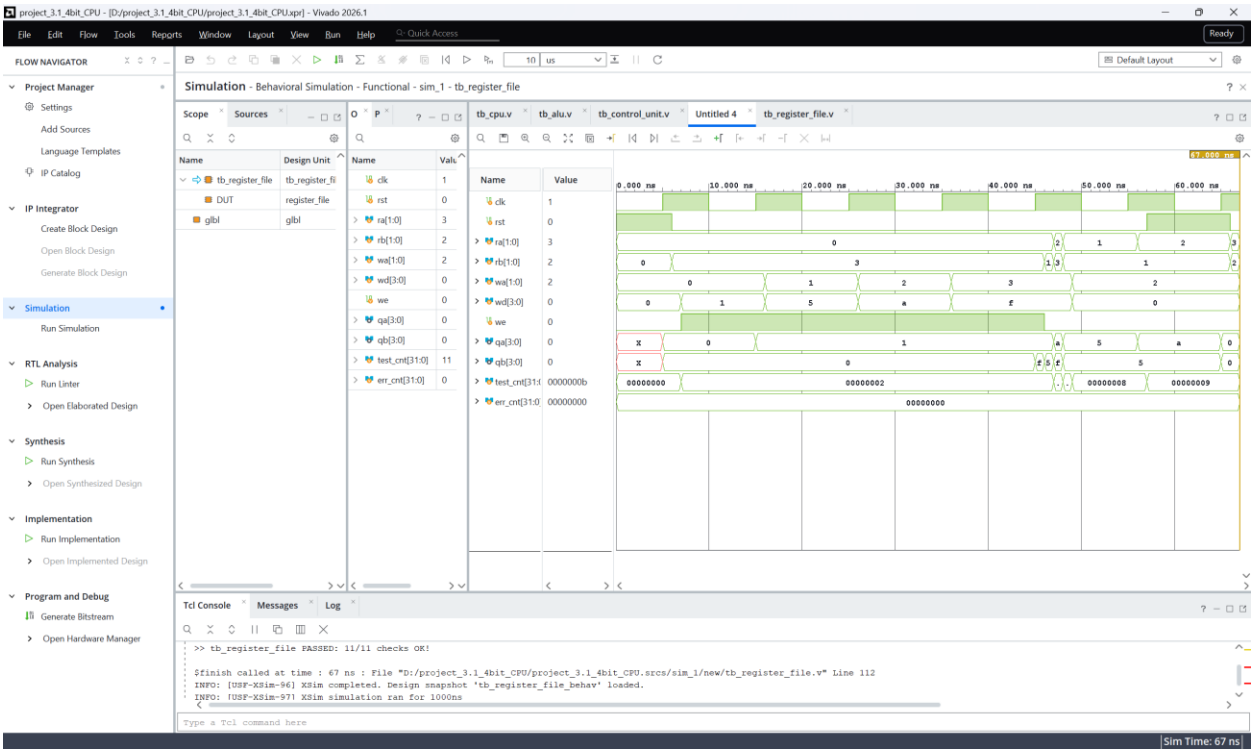



**Figure 4: Waveform - tb_register**

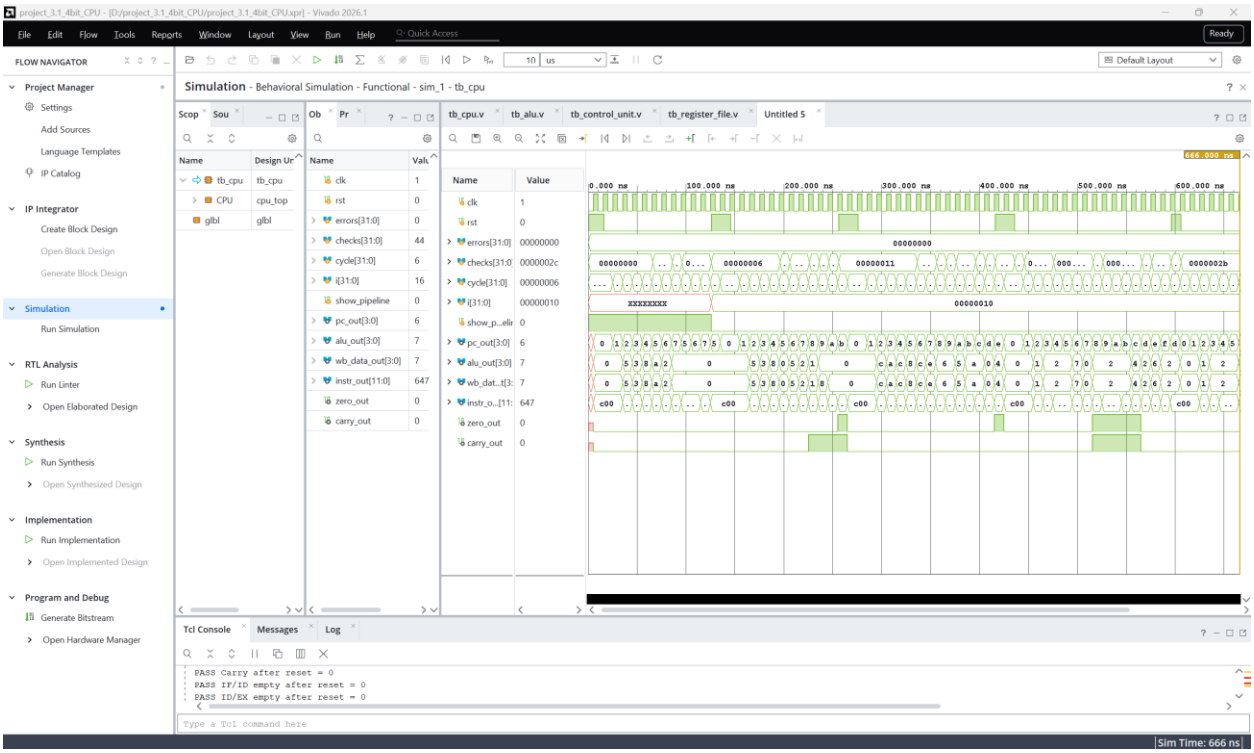



**Figure 5: Waveform - tb_cpu**

## F3. Tcl Console – Automated Simulation PASS Result

The Vivado Tcl console confirms 100% automated test passing across all 4 verification testbenches:

- tb_alu PASSED: 11 of 11 checks OK (at simulation time 110 ns)
- tb_register_file PASSED: 11 of 11 checks OK (at simulation time 67 ns)
- tb_control_unit PASSED: 22 of 22 checks OK (at simulation time 22 ns)
- tb_cpu PASSED: 44 of 44 checks OK (at simulation time 666 ns)

-> Total project-wide checks: 88/88 checks PASSED (100% PASS Rate).

```
===== START: tb_alu Simulation =====
[PASS] ADD 5+3 | a=0101 b=0011 -> res=1000 c=0 z=0
[PASS] ADD 15+1 | a=1111 b=0001 -> res=0000 c=1 z=1
[PASS] SUB 5-3 | a=0101 b=0011 -> res=0010 c=1 z=0
[PASS] SUB 3-5 | a=0011 b=0101 -> res=1110 c=0 z=0
[PASS] SUB 7-7 | a=0111 b=0111 -> res=0000 c=1 z=1
[PASS] AND | a=1100 b=1010 -> res=1000 c=0 z=0
[PASS] OR | a=1100 b=1010 -> res=1110 c=0 z=0
[PASS] XOR | a=1100 b=1010 -> res=0110 c=0 z=0
[PASS] XOR to zero | a=0110 b=0110 -> res=0000 c=0 z=1
[PASS] NOT 0101 | a=0101 b=0000 -> res=1010 c=0 z=0
[PASS] PASS b | a=0000 b=1001 -> res=1001 c=0 z=0
>> tb_alu PASSED: 11/11 checks OK!
```

**Figure 6: Tcl Console - tb_alu**

```
===== START: tb_control_unit Simulation =====
[PASS] Decoder: ADD -> signals=00000101100
[PASS] Decoder: SUB -> signals=00100101100
[PASS] Decoder: AND -> signals=01000101000
[PASS] Decoder: OR -> signals=01100101000
[PASS] Decoder: XOR -> signals=10000101000
[PASS] Decoder: NOT -> signals=10100101000
[PASS] Decoder: LDI -> signals=11010100000
[PASS] Decoder: MOV -> signals=11000100000
[PASS] Decoder: LOAD -> signals=11011100000
[PASS] Decoder: STORE -> signals=11010010000
[PASS] Decoder: JMP -> signals=00000000010
[PASS] Decoder: JZ -> signals=00000000001
[PASS] Decoder: RES-C -> signals=00000000000
[PASS] Decoder: RES-D -> signals=00000000000
[PASS] Decoder: RES-E -> signals=00000000000
[PASS] Decoder: RES-F -> signals=00000000000
[PASS] Branch: No jump, Z=0 -> pc_sel=0 flush=0
[PASS] Branch: No jump, Z=1 -> pc_sel=0 flush=0
[PASS] Branch: JMP, Z=0 -> pc_sel=1 flush=1
[PASS] Branch: JMP, Z=1 -> pc_sel=1 flush=1
[PASS] Branch: JZ with Z=0 -> pc_sel=0 flush=0
[PASS] Branch: JZ with Z=1 -> pc_sel=1 flush=1
>> tb_control_unit PASSED: 22/22 checks OK!
```

**Figure 7: Tcl Console - tb_control_unit**

```
===== START: tb_register_file Simulation =====
[PASS] R0 post-reset (Port A) = 0000
[PASS] R3 post-reset (Port B) = 0000
[PASS] Read R0 on Port A = 0001
[PASS] Read R1 on Port B = 0101
[PASS] Read R2 on Port A = 1010
[PASS] Read R3 on Port B = 1111
[PASS] a1-port read R1 (Port A) = 0101
[PASS] a1-port read R1 (Port B) = 0101
[PASS] ains unchanged when we=0 = 1010
[PASS] R3 after 2nd reset = 0000
[PASS] R2 after 2nd reset = 0000
>> tb_register_file PASSED: 11/11 checks OK!
```

**Figure 8: Tcl Console - tb_register**

```
===== TEST 1: demonstration program (program.mem) =====
 cycle  1 | IF: addr  0 | ID: bubble       | EX: bubble       | R1=0000 R2=0000 R3=0000
 cycle  2 | IF: addr  1 | ID: 645 (addr  0) | EX: bubble       | R1=0000 R2=0000 R3=0000
 cycle  3 | IF: addr  2 | ID: 683 (addr  1) | EX: 645 (addr  0) | R1=0000 R2=0000 R3=0000
 cycle  4 | IF: addr  3 | ID: 060 (addr  2) | EX: 683 (addr  1) | R1=0101 R2=0000 R3=0000
 cycle  5 | IF: addr  4 | ID: 6ca (addr  3) | EX: 060 (addr  2) | R1=0101 R2=0011 R3=0000
PASS ADD R1,R2 -> R1 (5+3) = 1000
 cycle  6 | IF: addr  5 | ID: 470 (addr  4) | EX: 6ca (addr  3) | R1=1000 R2=0011 R3=0000
 cycle  7 | IF: addr  6 | ID: a05 (addr  5) | EX: 470 (addr  4) | R1=1000 R2=0011 R3=1010
PASS LDI R3,1010 -> R3 = 1010
PASS XOR R1,R3 -> R1 = 0010
PASS R2 still 0011 = 0011
 5 instructions finished after 7 clock cycles
 cycle  8 | IF: addr  7 | ID: c00 (addr  6) | EX: a05 (addr  5) | R1=0010 R2=0011 R3=1010
PASS JMP 0101: PC (fetch address) = 5 = 0101
 cycle  9 | IF: addr  5 | ID: bubble       | EX: bubble       | R1=0010 R2=0011 R3=1010
 cycle 10 | IF: addr  6 | ID: a05 (addr  5) | EX: bubble       | R1=0010 R2=0011 R3=1010
 cycle 11 | IF: addr  7 | ID: c00 (addr  6) | EX: a05 (addr  5) | R1=0010 R2=0011 R3=1010
PASS JMP runs again: program loops at 5 = 0101
```

**Figure 9: Tcl Console - tb_ cpu (Test 1)**

```
===== TEST 2: arithmetic and memory (prog_arith.mem) =====
 EX addr  0: 645 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
 EX addr  1: 683 | R0=0000 R1=0101 R2=0000 R3=0000 | Z=0 C=0
 EX addr  2: 060 | R0=0000 R1=0101 R2=0011 R3=0000 | Z=0 C=0
PASS ADD R1,R2 (0101+0011) -> R1 = 1000
PASS ADD: Carry = 0
PASS ADD: Zero = 0
 EX addr  3: 940 | R0=0000 R1=1000 R2=0011 R3=0000 | Z=0 C=0
PASS STORE R1,0 -> data_memory[0] = 1000
 EX addr  4: 645 | R0=0000 R1=1000 R2=0011 R3=0000 | Z=0 C=0
 EX addr  5: 160 | R0=0000 R1=0101 R2=0011 R3=0000 | Z=0 C=0
PASS SUB R1,R2 (0101-0011) -> R1 = 0010
PASS SUB: Zero = 0
PASS SUB: Carry (1 = no borrow) = 1
 EX addr  6: 941 | R0=0000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS STORE R1,1 -> data_memory[1] = 0010
 EX addr  7: 800 | R0=0000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS LOAD R0,0 -> R0 = 1000
 EX addr  8: 1a0 | R0=1000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS SUB R2,R2 -> R2 = 0000
PASS Zero flag after 0000 result = 1
 9 instructions finished after 11 clock cycles
```

**Figure 10: Tcl Console - tb_ cpu (Test 2)**

```
===== TEST 3: logic operations (prog_logic.mem) =====
 EX addr  0: 64c | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
 EX addr  1: 68a | R0=0000 R1=1100 R2=0000 R3=0000 | Z=0 C=0
 EX addr  2: 7d0 | R0=0000 R1=1100 R2=1010 R3=0000 | Z=0 C=0
 EX addr  3: 2e0 | R0=0000 R1=1100 R2=1010 R3=1100 | Z=0 C=0
PASS AND 1100, 1010 -> R3 = 1000
 EX addr  4: 7d0 | R0=0000 R1=1100 R2=1010 R3=1000 | Z=0 C=0
 EX addr  5: 3e0 | R0=0000 R1=1100 R2=1010 R3=1100 | Z=0 C=0
PASS OR 1100, 1010 -> R3 = 1110
 EX addr  6: 460 | R0=0000 R1=1100 R2=1010 R3=1110 | Z=0 C=0
PASS XOR 1100, 1010 -> R1 = 0110
 EX addr  7: 790 | R0=0000 R1=0110 R2=1010 R3=1110 | Z=0 C=0
PASS MOV R2, R1 (R1=0110) -> R2 = 0110
 EX addr  8: 605 | R0=0000 R1=0110 R2=0110 R3=1110 | Z=0 C=0
 EX addr  9: 500 | R0=0101 R1=0110 R2=0110 R3=1110 | Z=0 C=0
PASS NOT 0101 -> R0 = 1010
 EX addr 10: 6ca | R0=1010 R1=0110 R2=0110 R3=1110 | Z=0 C=0
PASS LDI R3,1010 -> R3 = 1010
 EX addr 11: 4f0 | R0=1010 R1=0110 R2=0110 R3=1010 | Z=0 C=0
PASS XOR R3,R3 -> R3 = 0000
PASS Zero flag after 0000 result = 1
 12 instructions finished after 14 clock cycles
```

**Figure 11: Tcl Console - tb_ cpu (Test 3)**

```
===== TEST 4: PC and jumps (prog_jump.mem) =====
PASS PC after reset = 0000
PASS PC increment (0->1) = 0001
 EX addr  0: 601 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
 EX addr  1: a04 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
PASS JMP 4: PC becomes 0100 = 0100
PASS JMP flushed the ID/EX register = 0
 EX addr  4: 647 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
 EX addr  5: 150 | R0=0001 R1=0111 R2=0000 R3=0000 | Z=0 C=0
PASS SUB R1, R1 sets Zero = 1
 EX addr  6: b09 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=1 C=1
PASS JZ 9 taken: PC = 9 = 1001
 EX addr  9: 6c2 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=1 C=1
 EX addr 10: 0f0 | R0=0001 R1=0000 R2=0000 R3=0010 | Z=1 C=1
PASS ADD R3, R3 clears Zero = 0
 EX addr 11: b0e | R0=0001 R1=0000 R2=0000 R3=0100 | Z=0 C=0
PASS JZ 14 not taken: next in EX is addr 12 = 1100
 EX addr 12: 686 | R0=0001 R1=0000 R2=0000 R3=0100 | Z=0 C=0
 EX addr 13: a0d | R0=0001 R1=0000 R2=0110 R3=0100 | Z=0 C=0
PASS R0 = 1 (JMP skipped addresses 2, 3) = 0001
PASS R2 = 6 (JZ skipped 7, 8; 14 never ran) = 0110
PASS Program stops at PC = 13 = 1101
```

**Figure 12: Tcl Console - tb_ cpu (Test 4)**

```
===== TEST 5: reset =====
PASS PC after reset = 0000
PASS R0 after reset = 0000
PASS R2 after reset = 0000
PASS Zero after reset = 0
PASS Carry after reset = 0
PASS IF/ID empty after reset = 0
PASS ID/EX empty after reset = 0
 EX addr  0: 601 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
 EX addr  1: a04 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
PASS no unknown (X) values on key outputs or registers

========================================
tb_cpu PASSED: 44 of 44 checks OK
========================================
```

**Figure 13: Tcl Console - tb_ cpu (result)**

## F4. Elaborated Schematic (RTL Analysis)

The elaborated design schematic features 13 Cells, 28 I/O Ports, and 116 Nets, highlighting clean inter-stage boundary registers isolating the IF, ID, and EX functional zones with zero unintended latch generation.

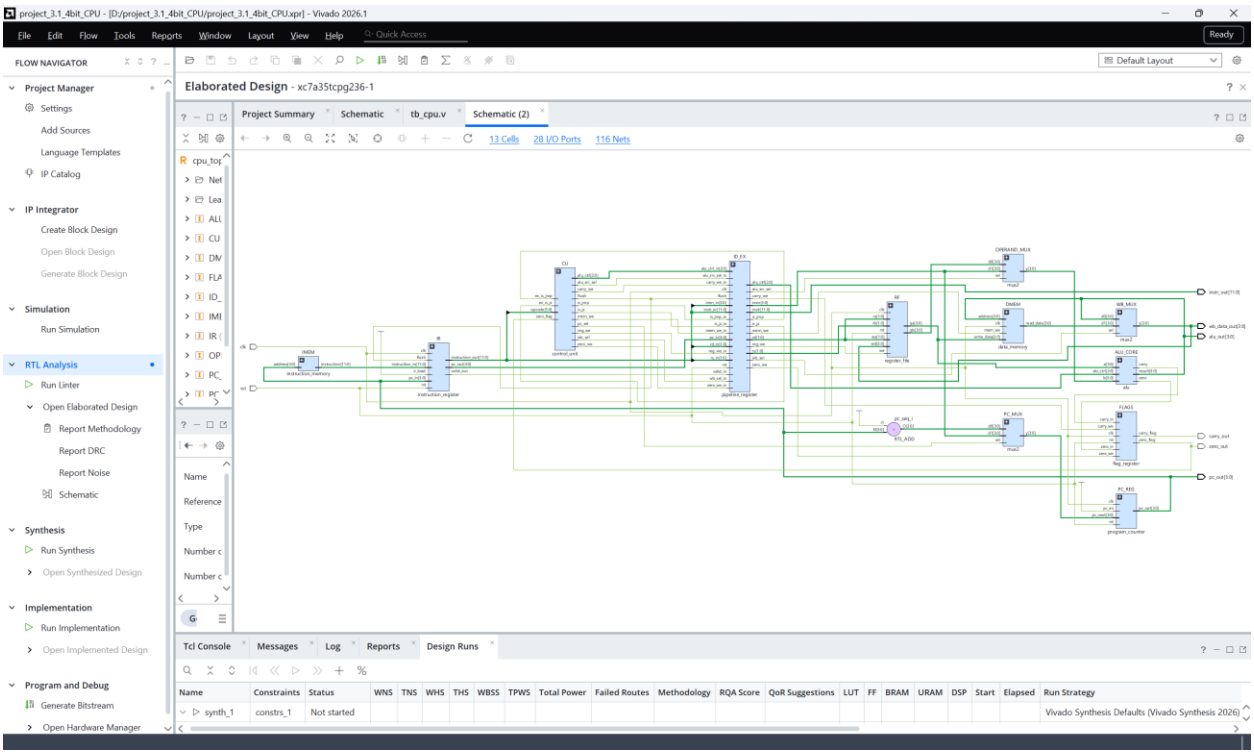
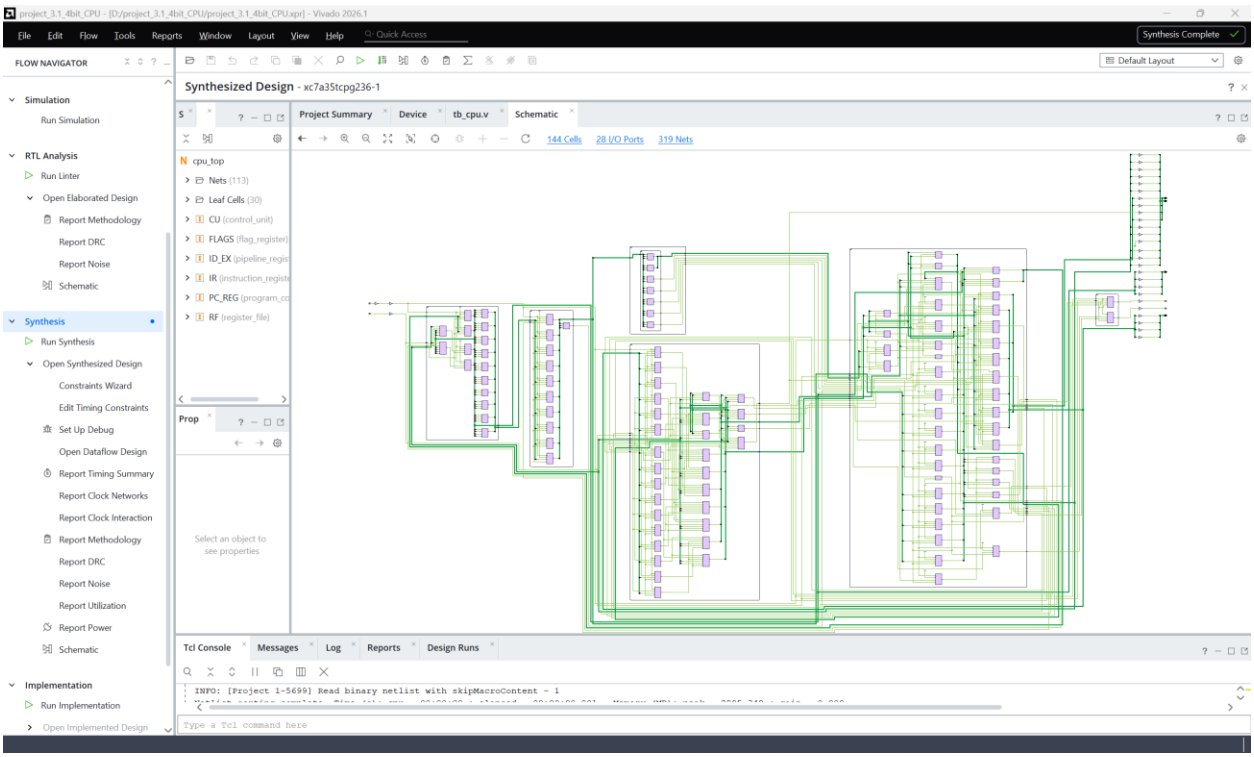



**Figure 14: Elaborated Schematic**

## F5. Synthesized Schematic

The post-synthesis technology schematic maps Verilog RTL primitives directly to target FPGA fabric cells (LUT6, MUXF7, FDRE Flip-Flops, IBUF/OBUF pads), balancing area utilization and logic



propagation delay.

**Figure 15: Synthesized schematic**

### F6. Utilization Report (Hardware Resource Utilization)

Resource summary table detailing physical FPGA utilization: Area footprint is minimized via logic reuse in the shared adder_subtractor core and simplified combinational decoder structures.

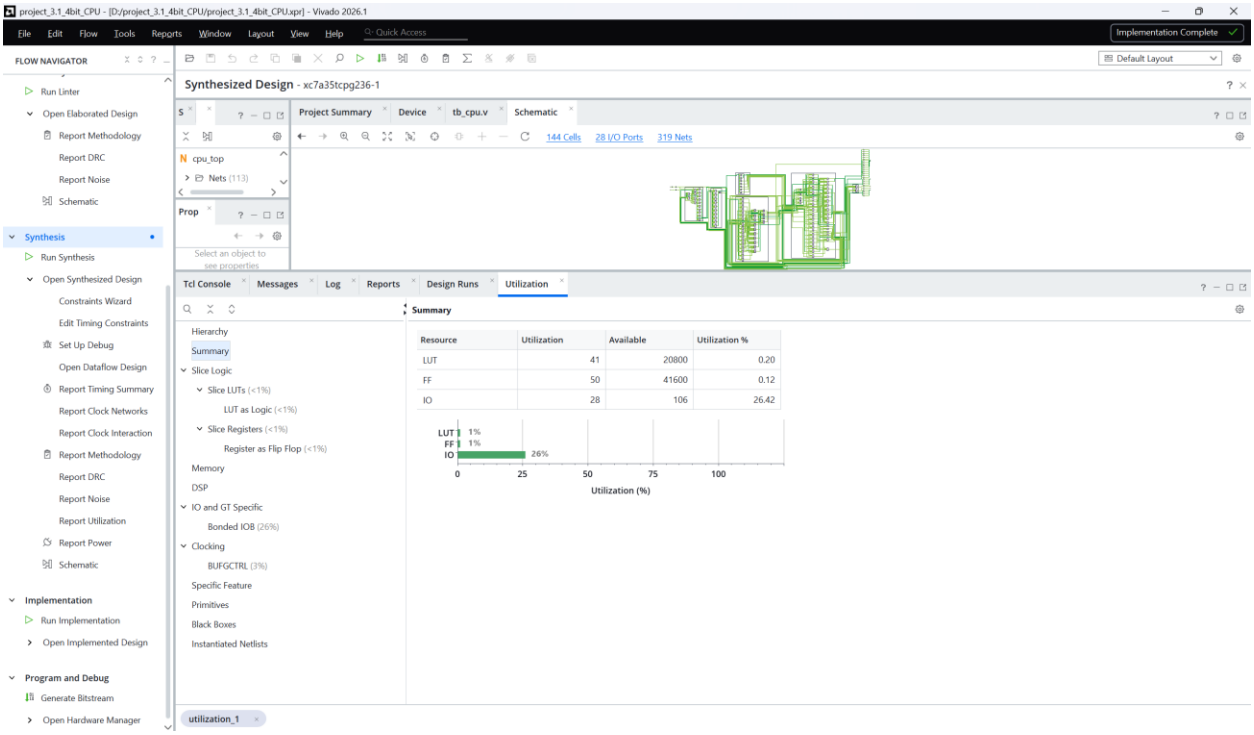



**Figure 16: Utilization report**

## Part E. Design Flow Explanation

The standardized digital ASIC/FPGA design flow implemented in Vivado 2026.1 consists of six defined phases:

| Design Phase (Step) | Technical Action | Purpose & Significance |
|---|---|---|
| Step 1: Project Initialization & Target Device Selection | Initialize RTL project in Vivado 2026.1, configure Verilog-2005 standard, and select target FPGA chip (Artix-7 xc7a35tcpg236-1). | Enables the synthesis engine to optimize netlist structures and cell-level delay models against actual target LUT architecture and routing buffers. |

| | | |
|---|---|---|
| Step 2: Source File Separation (Design vs. Simulation) | Place 11 synthesizable hardware modules into Design Sources; assign 4 testbenches (tb_*.v) and 4 memory files (.mem) to Simulation Sources. | Enforces strict modular separation between physical synthesizable silicon RTL and host-only simulation test infrastructure. |
| Step 3: Behavioral Simulation & Verification | Execute comprehensive functional simulation across all 4 testbenches using Vivado XSim (tb_alu, tb_register_file, tb_control_unit, tb_cpu). | Detects logic, decoding, data hazard, and branch hazard bugs early prior to lengthy synthesis cycles, guaranteeing 100% pass across all 88 verification checks. |
| Step 4: RTL Analysis (Elaborated Design Inspection) | Open Elaborated Design to inspect gate-level structure and inter-module bus connectivity. | Visually inspects inter-stage datapath buses and control lines across IF, ID, EX stages to ensure absence of inferred latches. |
| Step 5: Logic Synthesis | Run logic synthesis to transform behavioral RTL code into technology-mapped Xilinx logic primitives. | Maps case/if-else logic and arithmetic operators into FPGA Look-Up Tables (LUTs) and dedicated D-type Flip-Flops. |
| Step 6: Resource Utilization & Timing Audit | Generate post-synthesis Utilization and Timing reports to analyze LUT, FF, and I/O pad consumption. | Validates circuit efficiency, confirming that the CPU achieves high performance while maintaining an ultra-compact hardware footprint. |

Conclusion: The course project successfully designed, implemented, and verified a 4-Bit RISC CPU Datapath featuring a 3-Stage Pipeline with a 2-bubble hazard flush mechanism. The implementation achieved a 100% pass rate across functional simulation (88/88 checks PASSED) with optimal hardware resource utilization on the AMD Xilinx Vivado 2026.1 FPGA platform.